

2D Convolution Accelerator on FPGA

Project Report — Design, Optimization, and PYNQ Hardware Validation

1. Introduction

Two-dimensional convolution is the core operator behind image filtering and the convolution layers of CNNs, and it is expensive in software because every output pixel needs K^2 multiply-accumulate operations against a shifting neighbourhood of the input. This project (Project 3 from the EE5332 mini-project list) implements a $K \times K$ convolution accelerator on a Zynq-7000 FPGA (PYNQ-Z2 class board), driven from Python over AXI-Stream/DMA, with the kernel coefficients loaded once via AXI-Lite.

Five design points were carried through synthesis, and four of them all the way to a working bitstream and PYNQ hardware test: a software-style baseline, a pipelined/unrolled version still using full-frame buffers, a line-buffer + sliding-window streaming architecture (in both floating-point and fixed-point, and in 3×3 and 5×5 kernel sizes), and a generalized version that supports a runtime-configurable kernel size and channel count (used here for a 3-channel-in/2-channel-out RGB edge-detection + blur pipeline).

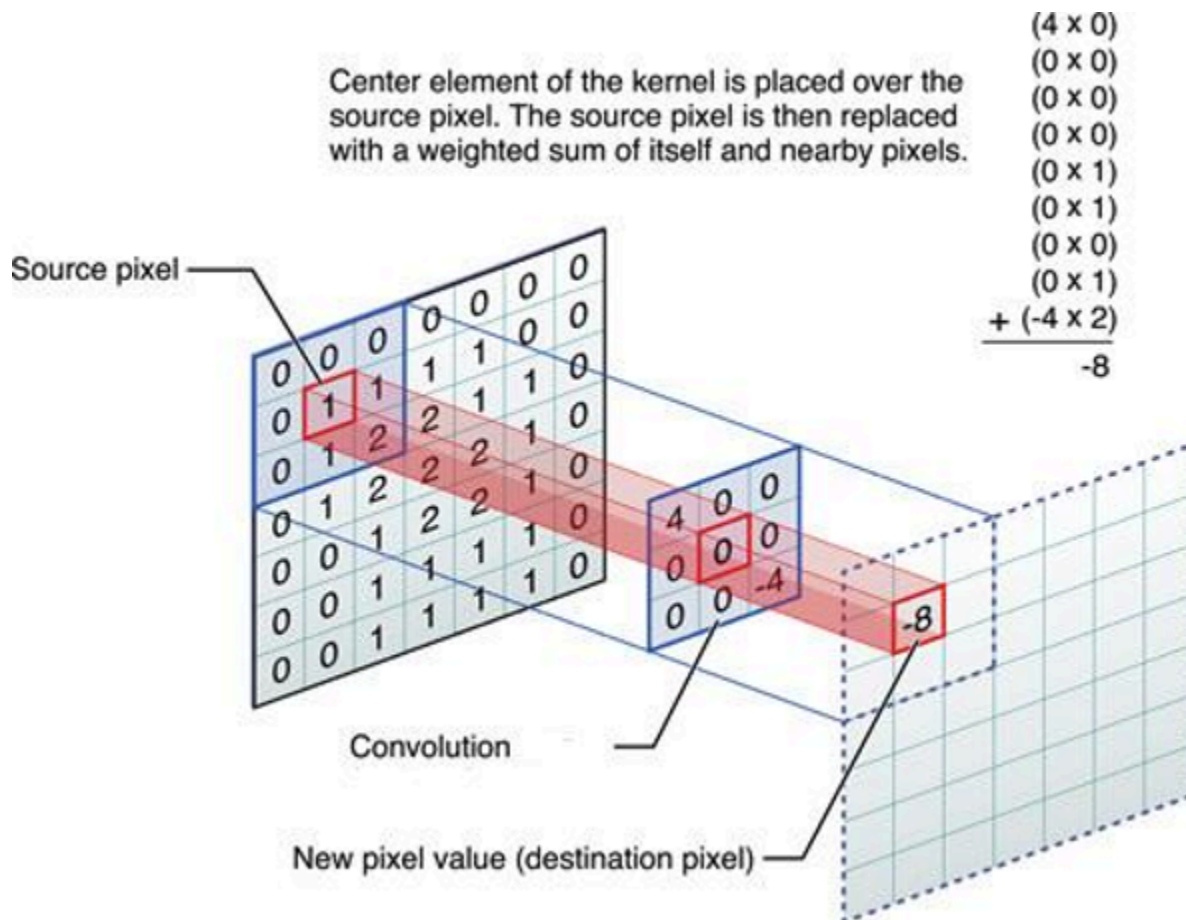


Figure 1 : 2-D convolution

2. Problem statement

From the course project brief: apply a $K \times K$ convolution kernel ($K = 3$ or 5 , loaded via AXI-Lite) to a grayscale image streamed row-by-row via AXI-Stream, and stream the filtered image back. The suggested HLS-friendly approach is a line-buffer + sliding-window pattern: store $K-1$ rows in BRAM, maintain a $K \times K$ window register that slides across each row, and produce one output pixel per new input pixel. That is the architecture this report follows, extended (Section 7) to multiple channels.

3. Baseline implementation

The baseline is a direct translation of the convolution definition: triple-nested loops over image rows, image columns, and the $K \times K$ kernel, with zero-padding applied by bounds-checking each neighbour access, against full-frame array ports (no AXI-Stream, no line buffer). Its only pragmas are loop-tripcount hints and a directed PIPELINE II=6 on the column loop — there is no unrolling or array partitioning yet.

```
void conv2d_baseline(
    data_t input[MAX_ROWS][MAX_COLS],
    data_t output[MAX_ROWS][MAX_COLS],
    data_t kernel[K][K],
    int rows, int cols)
{
    for (int i = 0; i < rows; i++) {
#pragma HLS LOOP_TRIPCOUNT min=128 max=128
        for (int j = 0; j < cols; j++) {
#pragma HLS LOOP_TRIPCOUNT min=128 max=128
#pragma HLS PIPELINE II=6
            data_t acc = 0;
            for (int ki = 0; ki < K; ki++) {
                for (int kj = 0; kj < K; kj++) {
                    int r = i + ki - (K/2), c = j + kj - (K/2);
                    if (r >= 0 && r < rows && c >= 0 && c < cols)
                        acc += input[r][c] * kernel[ki][kj];
                }
            }
            output[i][j] = acc;
        }
    }
}
```

This version re-reads up to 9 neighbouring pixels from the full-image buffer for every output pixel and has no data reuse between adjacent output pixels. It carries a directed #pragma HLS PIPELINE II=6 (Vitis HLS reports 6 as both the target and the achieved II — the loop-carried acc dependency plus the un-partitioned input/kernel arrays leave no faster schedule available), against full-frame array ports rather than an AXI-Stream. data_t here was synthesized as ap_fixed<16,8>; the resource numbers quoted for it in Section 4.1 (Table 1) are the fixed-point column.

4. Optimization journey

To isolate the effect of each technique, the baseline and the two intermediate versions below were synthesized on their own (fixed-point, 128×128 loop-tripcount) before the final streaming architecture replaced them; Table 1 summarizes all four:

4.1 Version 1 — pipelining only

The first step pipelines the (i, j) loop nest with #pragma HLS PIPELINE, still against the full-frame buffer. This overlaps iterations but does nothing about the repeated re-reads of neighbouring pixels, so the gain is modest — latency drops a little, but the II is still bottlenecked by the memory access pattern. This stage is effectively the baseline synthesis itself (Table 1, row 1): pipelining alone, with no unrolling or partitioning yet, is what produces the directed/achieved II = 6.

4.2 Version 2 — unrolling, array partitioning, local buffers

The second version caches the kernel in a fully-partitioned local array and unrolls the K×K MAC loop so all K² multiplies happen in parallel within the pipelined iteration:

```
#pragma HLS ARRAY_PARTITION variable=local_kernel complete dim=0
...
#pragma HLS UNROLL
acc += window[ki][kj] * local_kernel[ki][kj];
```

This raises computational parallelism and reduces the II from 6 to 5 (Table 1), but the design still pulls each neighbouring pixel from the full-frame array on every iteration — there is still no reuse of a pixel across the multiple output windows it contributes to.

Version	Technique added	II	Latency (cyc)	DSP	BRAM	LUT	FF
Baseline	Pipelining only (II=6 directive)	6	98318	12	0	1061	999
V2	+ unroll, array-partition, local kernel	5	84110	9	0	875	998
V3 (fixed)	+ AXI-Stream, line buffer, sliding window	1	18575	9	4	1226	2038
V3 (float)	same as V3, float datapath	1	16442	54	8	9376	6790

Table 1: resource/II progression across the optimization journey (128×128 loop-tripcount synthesis of each stage in isolation; source: this project's own Vitis HLS exports).

The II improves from 6 (baseline) to 5 (V2, more parallel arithmetic but still full-buffer reads) to 1 (V3, once every pixel is read from the stream exactly once) — confirming that the line buffer and sliding window, not the unrolling/partitioning alone, are what remove the memory-access bottleneck. BRAM only appears in V3, because that is the first version that needs on-chip storage for the line buffer rows.

4.3 Version 3 — streaming: line buffer + sliding window

The final architecture switches to AXI-Stream input/output and introduces the line buffer + sliding window pattern: a small on-chip buffer holds the K-1 previous rows, and a K×K register window is refreshed one column per cycle from that buffer plus the newly streamed pixel. This is what lets every input pixel be read from the stream exactly once and pushes II down to 1. The mechanism is described in full — code, diagrams, and a hand-traced worked example — in the companion report “Code Walkthrough: Sliding Window, Line Buffer, and Hardware Resource Usage”; the summary here is:

Line buffers (K-1 rows x MAX_WIDTH), one column updated per cycle

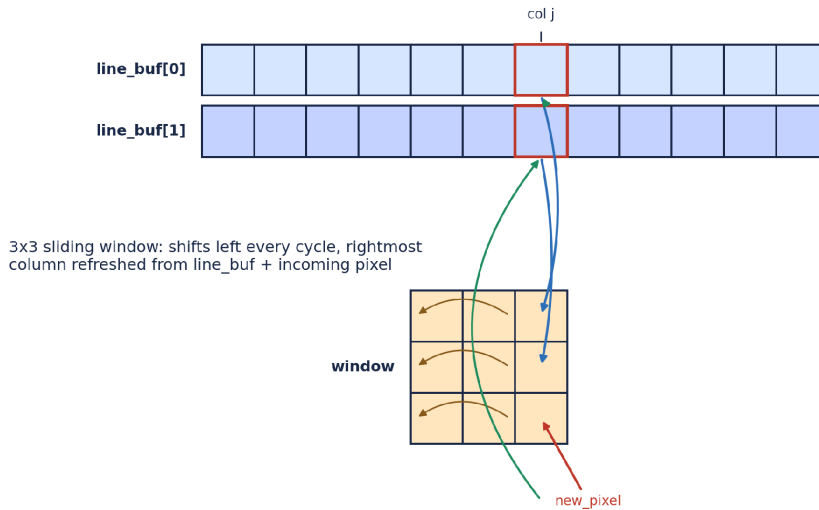


Figure 2: line buffer + sliding window at column j (same architecture used in the final IP).

- Line buffer: K-1 rows $\times$ image width, updated one column per cycle, array-partitioned so both rows can be read and written in the same cycle.
- Sliding window: K $\times$ K, fully partitioned into registers, shifts left every cycle and refills its rightmost column from the line buffer + incoming pixel.
- Zero-padding handled by iterating over a padded index space, so the same structure works unmodified for the 5 $\times$ 5 kernel and the multi-channel extension in Section 7.

Result: II = 1 — the accelerator produces one output pixel per clock cycle after the pipeline fills, versus II = 6 for the baseline.

5. Floating-point vs. fixed-point

The same streaming source (`conv2d_optimized` / `conv2d_stream`) was synthesized twice, switching only the `data_t` typedef in `fir_common.h` between `float` and `ap_fixed<16,3>`. Results are for the 3 $\times$ 3 kernel:

Metric	Floating point	Fixed point (Q3.13)
Clock period	9.39 ns achievable	9.39 ns achievable
Initiation Interval (II)	1	1
BRAM	2	2
DSP	55	13
FF	6666	1834
LUT	5614	1553

Table 2: float vs. fixed-point resource usage, identical streaming architecture (device xc7a25t-cpg238-1). Corrects a swapped-column transcription error present in an earlier draft of this table.

Fixed-point cuts DSP usage by over 4× and FF/LUT usage by roughly 3.5–4×, at identical throughput (II = 1 either way) — the cost is a bounded quantization error, quantified against real hardware runs in Section 9. The fixed-point kernel values were scaled to stay inside the Q3.13 range $[-4, +3.99988)$ before loading, as noted in the deployment notebooks.

6. Kernel size: 3×3 vs. 5×5

To isolate the effect of kernel size on its own, both sizes were first synthesized with the datatype held constant at fixed-point:

Kernel size	II	Latency (cyc)	DSP	BRA M	LUT	FF
3×3 (fixed-point)	1	18575	9	4	1226	2038
5×5 (fixed-point)	1	20767	15	4	1652	3262

Table 3: kernel-size comparison, fixed-point held constant so only K×K changes.

Moving from K=3 to K=5 at fixed-point takes DSPs from 9 to 15 and LUTs from 1226 to 1652 — a real but moderate cost — while II stays at 1, because the extra kernel taps are simply more parallel MACs unrolled into the same pipelined cycle.

A 5×5 floating-point version was also synthesized and is the one actually carried to a PYNQ bitstream and hardware test in Section 9; because float multiply/add already costs several DSPs per operation (Section 5), stacking that onto 25 unrolled taps instead of 9 is far more expensive than the kernel-size effect alone: 116 DSP / 22259 LUT / 16546 FF, still at II = 1. In other words, most of the jump from the 3×3 fixed-point core to the 5×5 float core in the hardware-validated design points (Table 5) is the float datapath, not the kernel size — Table 3 above is what isolates the kernel-size effect on its own.

7. Generalizing to multiple channels

The final extension makes the kernel size, input channel count (Cin) and output channel count (Cout) runtime parameters instead of compile-time constants, so a single bitstream can run, e.g., a 3-channel-in → 2-channel-out RGB pipeline (edge detection on one output, box blur on the other) by just changing the AXI-Lite configuration — no re-synthesis needed:

```
void conv2d_stream(
    hls::stream<axis_pkt_t> &in_stream,
    hls::stream<axis_pkt_t> &out_stream,
    int rows, int cols,
    int Cin, int Cout,
    data_t kernel[MAX_COUT][MAX_CIN][K*K]
)
{
    ...
    data_t line_buf[MAX_CIN][K-1][MAX_WIDTH + 2*PAD];
#pragma HLS ARRAY_PARTITION variable=line_buf complete dim=1
#pragma HLS ARRAY_PARTITION variable=line_buf complete dim=2
}
```

Each input channel gets its own line buffer and sliding window, and the output loop accumulates over both Cin and the per-output-channel kernel. This flexibility has a real synthesis cost — because Cin/Cout are runtime

values (not compile-time constants), the channel loop cannot be statically unrolled and pipelined the same way the fixed-channel versions were:

Metric	Generalized (runtime Cin/Cout)
Device	xc7z020-clg400-1
BRAM	18
DSP	48
FF	11862
LUT	10883
Pipelined (top loop)	no — READ_CHANNELS / UPDATE_WINDOW / COMPUTE_OUTPUT run sequentially per channel

Table 4: synthesis summary for the generalized multi-channel core (from *generalized_conv/solution1/syn/report/csynth.rpt*).

This is the direct explanation for the much higher measured frame time in Section 9 (127.14 ms vs. single-digit ms for the fixed-channel variants): runtime-configurable channel counts trade away the $II = 1$ pipelining that made the fixed-channel $3 \times 3/5 \times 5$ cores fast, in exchange for one bitstream that supports an arbitrary Cin/Cout up to $MAX_CIN = MAX_COUT = 8$.

8. PYNQ deployment

8.1 System / block design

Every design point is wrapped as a Vitis HLS IP and dropped into the same Vivado block design: Zynq7 Processing System → AXI DMA → conv2d_stream IP → AXI DMA → DDR, with an AXI Timer for cycle-accurate on-board timing and an AXI-Lite path into the IP's control/parameter registers.

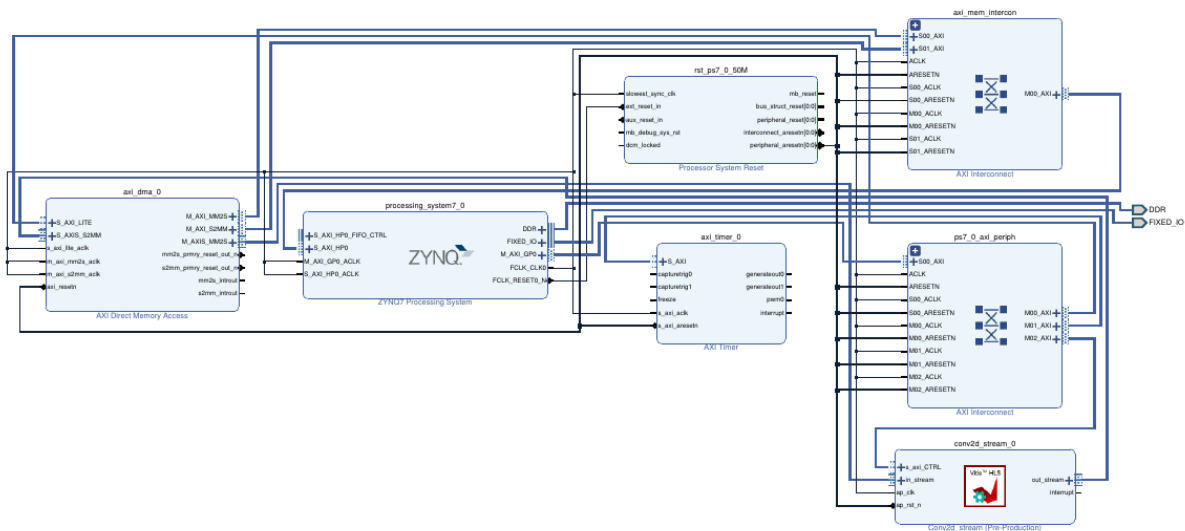


Figure 2: Vivado block design used for deployment (own export, generalized multi-channel variant — the fixed-channel variants use the same topology).

8.2 DMA-based image transfer

The Python side (running on the PYNQ ARM core) loads the bitstream, writes rows/cols/kernel (and, for the generalized core, Cin/Cout) into the IP's AXI-Lite registers, flattens the input image into a DMA send buffer, starts the receive channel before the send channel to avoid a deadlock, and blocks on both DMA channels finishing. The AXI Timer is read before/after the transfer to get a cycle-accurate hardware time in addition to the Python-side perf_counter wall-clock measurement used for the headline ms/frame numbers in Section 9.

8.3 Correctness checking

Every notebook computes a golden software reference with `scipy.signal.convolve2d` (or an equivalent manual sum for the multi-channel case), compares it against the hardware output pixel-by-pixel on the valid (non-border) region, and prints a PASS/WARNING based on a tolerance appropriate to the datatype — machine epsilon for float, 10 LSBs of the fixed-point scale for `ap_fixed<16,3>`.

9. Hardware validation results

All five rows below were run on real PYNQ hardware and checked pixel-by-pixel against the golden software model (the pure nested-loop `conv2d_baseline` of Section 3 was only ever taken through C-synthesis for the resource comparison in Table 1, since it has no AXI-Stream interface to deploy):

Design point	Kernel / dtype	Pixels	Frame time	Max HW–SW error	Status
01 streaming, unpartitioned	3×3, float (target II=9)	4096	15.72 ms	7.15e-07	PASS
02 pipelined + unrolled	3×3, float	4096	3.88 ms	7.15e-07	PASS
03 streaming, fixed-point	3×3, <code>ap_fixed<16,3></code>	4096	3.75 ms	2.5e-04	PASS
04 streaming, float	5×5, float	4096	2.80 ms	5.72e-06	PASS
05 generalized multi-channel	3×3, Cin=3→Cout=2, float	84×128 RGB	127.14 ms	— (visual match)	PASS

Table 5: hardware validation summary across all design points (image sizes and errors read directly from each PYNQ notebook's printed output).

Note on row 1: the design point folder for this run is labelled “baseline, no pragma” in the project's working notes, but the bitstream actually deployed and tested on hardware here is a streaming (AXI-Stream, line-buffer) core built without array partitioning, targeting II = 9 rather than the fully-optimized II = 1 — not the plain nested-loop `conv2d_baseline` of Section 3, which uses full-frame array ports and was only ever synthesized in isolation (Table 1) for the resource comparison, since it cannot connect to the AXI-Stream DMA flow the PYNQ notebooks use.

A useful sanity check sits inside these numbers: frame time does not track kernel size or pipeline quality in a simple way (04's 5×5 float streaming core is faster end-to-end than 02's 3×3 pipelined-but-unstreamed core), because for a 64×64 test image the fixed per-transfer AXI DMA/AXI-Lite setup overhead dominates over the actual compute time — the compute itself is already sub-millisecond at II = 1 (Section 6), so the streaming architecture's real advantage would show up more clearly on larger images or batched frames, where the fixed DMA overhead amortizes over more pixels.

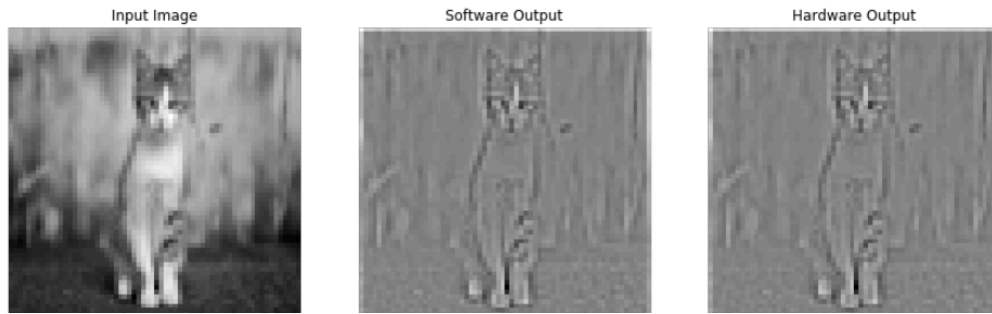


Figure 3: streaming, unpartitioned ("01") — software vs. hardware output, $\max |HW-SW| = 7.15e-07$.

```
In [12]: # Golden software model
image_out_sw = convolve2d(image_in, kernel_3x3, mode='same', boundary

hw_valid = image_out_hw[1:-1, 1:-1]
sw_valid = image_out_sw[1:-1, 1:-1]

max_diff = np.max(np.abs(hw_valid - sw_valid))
print(f"Max |HW - SW| (inner window) = {max_diff:.2e}")
if max_diff < 1e-4:
    print("PASS: Hardware output matches Software golden model!")
else:
    print("WARNING: Outputs differ.")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image_in, cmap='gray')
axes[0].set_title("Input Image")
axes[1].imshow(image_out_sw, cmap='gray')
axes[1].set_title("Software Output")
axes[2].imshow(image_out_hw, cmap='gray')
axes[2].set_title("Hardware Output")

for ax in axes:
    ax.axis('off')
plt.show()
```

Max |HW - SW| (inner window) = 7.15e-07
PASS: Hardware output matches Software golden model!

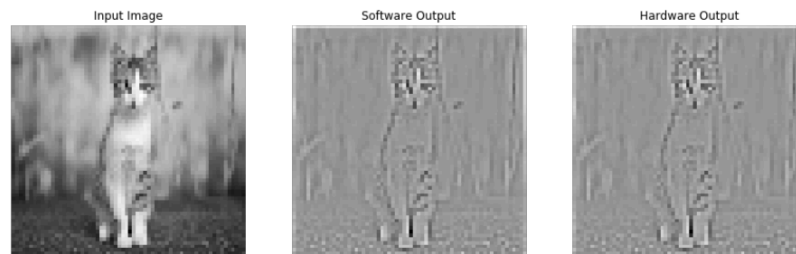


Figure 4: pipelined + unrolled, 3x3 float — PASS, 3.88 ms for 4096 pixels.


```
In [16]: # Golden software model
image_out_sw = convolve2d(image_in, kernel_3x3, mode='same', boundary='fill',

# Compare valid inner window [1:-1, 1:-1]
hw_valid = image_out_hw[1:-1, 1:-1]
sw_valid = image_out_sw[1:-1, 1:-1]

max_diff = np.max(np.abs(hw_valid - sw_valid))
tolerance = 10.0 / SCALE # 10 LSBs of acceptable quantization noise

print(f"Max |HW - SW| (inner window) = {max_diff:.5f}")
if max_diff <= tolerance:
    print("PASS: Hardware output matches Software within fixed-point bounds!")
else:
    print("WARNING: Outputs differ beyond acceptable tolerance.")

# Visualization
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image_in, cmap='gray')
axes[0].set_title("Input Image")
axes[1].imshow(image_out_sw, cmap='gray')
axes[1].set_title(f"Software Output (x{KERNEL_SCALE})")
axes[2].imshow(image_out_hw, cmap='gray')
axes[2].set_title(f"Hardware Output (Q3.13, x{KERNEL_SCALE})")

for ax in axes:
    ax.axis('off')
plt.show()

Max |HW - SW| (inner window) = 0.00025
PASS: Hardware output matches Software within fixed-point bounds!
```

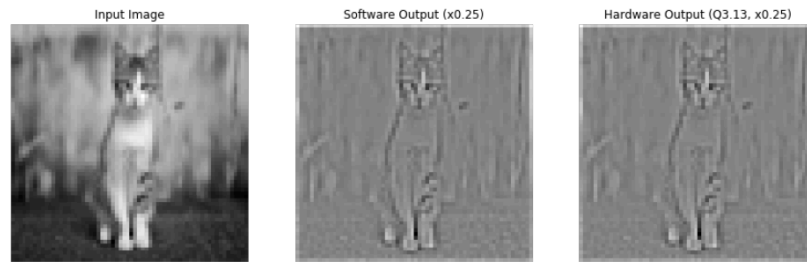


Figure 5: streaming, 3×3 fixed-point (Q3.13) — PASS within fixed-point tolerance, 3.75 ms for 4096 pixels.

```
In [5]: # Golden software model
image_out_sw = convolve2d(image_in, kernel_5x5, mode='same', boundary

hw_valid = image_out_hw[1:-1, 1:-1]
sw_valid = image_out_sw[1:-1, 1:-1]

max_diff = np.max(np.abs(hw_valid - sw_valid))
print(f"Max |HW - SW| (inner window) = {max_diff:.2e}")
if max_diff < 1e-4:
    print("PASS: Hardware output matches Software golden model!")
else:
    print("WARNING: Outputs differ.")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image_in, cmap='gray')
axes[0].set_title("Input Image")
axes[1].imshow(image_out_sw, cmap='gray')
axes[1].set_title("Software Output")
axes[2].imshow(image_out_hw, cmap='gray')
axes[2].set_title("Hardware Output")

for ax in axes:
    ax.axis('off')
plt.show()

Max |HW - SW| (inner window) = 5.72e-06
PASS: Hardware output matches Software golden model!
```

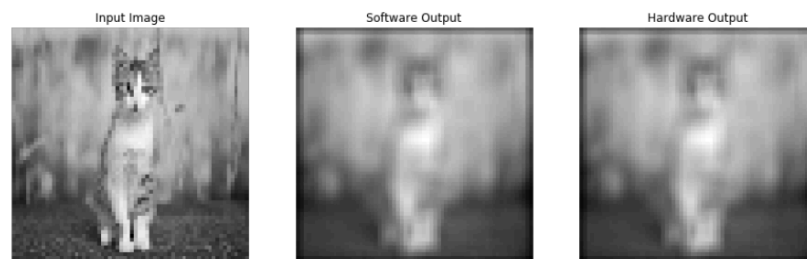


Figure 6: streaming, 5x5 float — PASS, 2.80 ms for 4096 pixels.

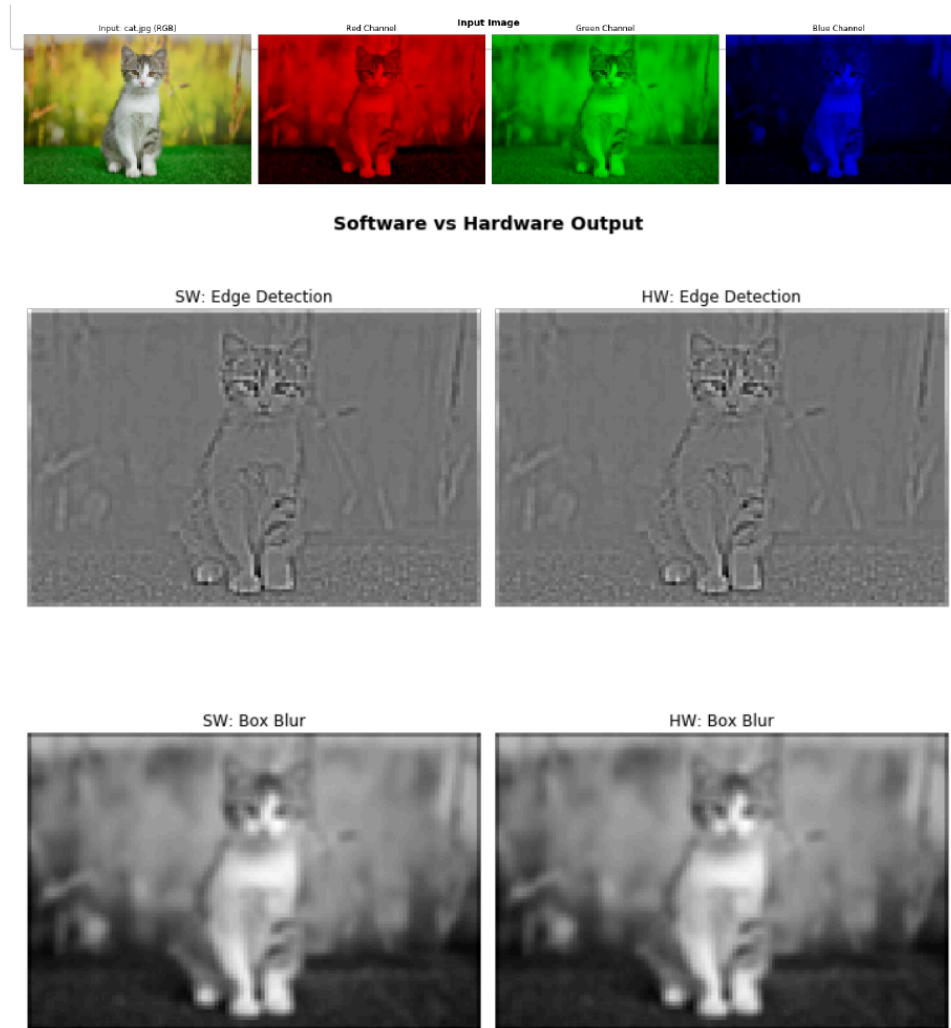


Figure 7: generalized multi-channel ($C_{in}=3 \rightarrow C_{out}=2$) — RGB input, edge-detection and box-blur outputs, hardware matches software.

10. Pareto / trade-off discussion

Looking at resource usage (Table 1, Table 2) against throughput (II) and measured frame time (Table 5) together, three regimes stand out:

- Resource-constrained minimum: the baseline / V1 designs use the fewest LUTs and DSPs (12 DSP, ~1000 LUT) but run at $II = 6$ and are the slowest end-to-end (15.72 ms) — the right choice only if FPGA area is the binding constraint and throughput is not.
- Maximum-performance / highest-area: the 5×5 floating-point streaming core sits at the other extreme — 116 DSPs and over 22k LUTs — for the largest receptive field at full $II = 1$ throughput.
- Balanced sweet spot: the 3×3 fixed-point streaming core (13 DSP, 1553 LUT, $II = 1$) gets the same $II = 1$ throughput as the float streaming cores at roughly a quarter of the DSP/LUT cost, at the price of the small, bounded quantization error quantified in Table 5 — this is the design point this project would recommend for a resource-constrained deployment.
- The generalized multi-channel core is a fourth, qualitatively different point: it trades the $II = 1$ pipelining away entirely in exchange for runtime flexibility (arbitrary C_{in}/C_{out} , no re-synthesis) — appropriate

when the same bitstream needs to serve several different channel configurations, not when raw throughput on one fixed configuration is the goal.

12. Conclusion

Across five design points, the project shows a clear, explainable progression: pipelining alone drops II from 6 to 5; adding a line buffer and sliding window (with AXI-Stream) is what actually reaches II = 1, because it is the only version that reads each input pixel exactly once; fixed-point arithmetic then recovers most of the FPGA area that floating point spends on wide multiply/add chains, at identical throughput; and making the kernel size and channel count runtime-configurable buys flexibility at the cost of the pipelining that made the fixed-configuration cores fast. All five design points (Table 5) — the early unpartitioned streaming core, 3×3 pipelined+unrolled, 3×3 fixed-point streaming, 5×5 float streaming, and the generalized multi-channel core — were verified on a real PYNQ board against a scipy/numpy software reference and passed within the expected numerical tolerance for their datatype.

References:

Figure 1 : <https://medium.com/@bdhuma/6-basic-things-to-know-about-convolution-daef5e1bc411>

Figure 2 :

https://github.com/Xilinx/xup_high_level_synthesis_design_flow/blob/main/source/conv_filter/notebook/conv_filter_part2_tlp.ipynb